

Approval process in Salesforce



Explain approval process in Salesforce in detail with multiple examples in detail

The **Approval Process** in **Salesforce** is an automated process that helps organizations **automate how records are approved** in Salesforce. It specifies **a sequence of steps** that are required to approve a record, who can approve them, and what actions are taken at each step.

What is an Approval Process?

An **Approval Process** is a **set of steps** that defines:

- When a record should be submitted for approval.
- Who must approve it at each step.
- What to do after each approval/rejection.

Key Components of an Approval Process

Component	Description
Entry Criteria	Conditions that decide whether a record enters the approval process.
Approval Steps	The sequence of approvals and who should approve them.
Initial Submission Actions	Actions triggered when the record is first submitted.
Approval Actions	Actions triggered when a step is approved.
Rejection Actions	Actions triggered when a step is rejected.
Final Approval Actions	Actions triggered when all steps are approved.
Final Rejection Actions	Actions triggered when any step is rejected.
Recall Actions	Actions triggered when a record is recalled by the submitter.

Real-World Example Scenarios

Example 1: Leave Request Approval

Use Case: An employee submits a leave request, which must be approved first by the manager and then by HR.

Object: Custom Object `Leave_Request__c`

Steps:

1. Entry Criteria:

- Record Status = "Submitted"

- Leave Days ≤ 15

2. Approval Steps:

- **Step 1:** Manager (Related via Manager field on User)
- **Step 2:** HR (Queue or specific user)

3. Initial Submission Actions:

- Send email to Manager
- Update status to "Pending Manager Approval"

4. Approval Actions (Step 1):

- Email HR
- Update status to "Pending HR Approval"

5. Approval Actions (Final Step):

- Update status to "Approved"
- Send confirmation email to employee

6. Rejection Actions:

- Update status to "Rejected"
- Notify employee with rejection reason

7. Final Approval Actions:

- Update Leave Balance
- Notify employee and HR

✓ Example 2: Opportunity Discount Approval

Use Case: A Sales Rep offers a discount $> 20\%$ on an Opportunity. It requires approval from the Sales Manager and then the Finance team.

Object: Opportunity

Steps:

1. Entry Criteria:

- `Discount__c` $> 20\%$

2. Approval Steps:

- **Step 1:** Manager of the Opportunity Owner
- **Step 2:** Finance Head

3. Initial Submission Actions:

- Lock the Opportunity Record (prevent edits)
- Send approval request to Manager

4. Approval Actions (Step 1):

- Send request to Finance Head

5. Final Approval Actions:

- Unlock record
- Update status to "Approved for Discount"
- Notify Sales Rep

6. Final Rejection Actions:

- Unlock record
- Set Discount back to 20%
- Notify Sales Rep

✅ Example 3: Purchase Request Approval Process

Use Case: Any purchase request over ₹1,00,000 needs two-level approval: Department Head and then Procurement Head.

Object: Custom Object `Purchase_Request__c`

Entry Criteria:

- Amount > 100000

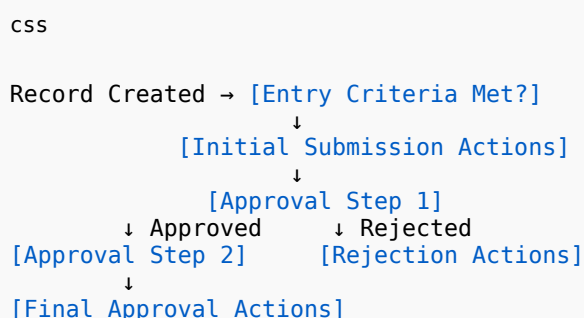
Approval Steps:

1. Department Head
2. Procurement Head

Actions:

- Notify approvers at each step
- Create a Task on approval/rejection
- Update status field to reflect progress

Approval Process Flow (Visual Representation)



Record Locking in Approval Process

Once a record is submitted:

- It is **locked** (no one can edit except admins or with special permission).
- You can unlock it automatically on approval/rejection using actions.

Tools to Define Approval Processes

- **Point-and-click** via **Setup** → **Approval Processes**
- Define via **Process Builder** or **Flow** to trigger submission
- Use **Apex** (`Approval.ProcessSubmitRequest`) for programmatic submission

Submission Methods

- Manual: User clicks **Submit for Approval**
- Automatic: Via **Process Builder** or **Apex Code**

Best Practices

- Always use **Queues** or **Roles** instead of hardcoding users.
- Keep approval steps **dynamic** using formulas or related user fields.
- Lock records only if necessary.
- Provide **clear rejection reasons** to users.
- Send **email notifications** or **Chatter posts** on status change.

Summary

Feature	Description
Multi-step approvals	Yes (supports sequential or parallel)
Conditional branching	No (requires multiple processes or Flows)
Integration with Apex	Yes
Record Locking	Yes
Notification via Email	Yes (configurable)
Recall/Resubmit Options	Yes

Would you like me to show you how to configure one of the above approval processes step-by-step in Salesforce Setup UI?



Yes

Great! Let's walk through **how to configure an Approval Process in Salesforce** step-by-step using the **Leave Request Approval** example we discussed earlier.

✓ Use Case Recap: Leave Request Approval

Object: `Leave_Request__c` (custom object)

Requirement:

- Step 1: Manager approves
- Step 2: HR approves
- Entry criteria: `Status__c = 'Submitted' AND Leave_Days__c ≤ 15`

🔧 Step-by-Step: Creating the Approval Process

Step 1: Create Custom Object and Fields (if not already present)

If you haven't created the object yet:

1. Go to **Setup** → **Object Manager** → **Create** → **Custom Object**

- Label: **Leave Request**
- API Name: `Leave_Request__c`

Add Fields:

- `Employee__c` (Lookup to User)
- `Manager__c` (Lookup to User)
- `Leave_Days__c` (Number)
- `Status__c` (Picklist: Submitted, Approved, Rejected, Pending Manager Approval, Pending HR Approval)
- `Reason__c` (Text Area)

Step 2: Create the Approval Process

1. Go to **Setup** → Search for **"Approval Processes"**
2. Select your object: `Leave_Request__c`
3. Click **"Create New Approval Process"** → **Use Jump Start Wizard** (or "Standard Setup Wizard" for full control)

Let's use the **Standard Wizard** for full customization.

Step 3: Define Process Properties

- Name: `Leave_Request_Approval`
- Unique Name: `Leave_Request_Approval`
- Description: Approval process for leave requests with ≤ 15 days.
- Entry Criteria:
 - Field: `Status__c` equals `Submitted`
 - AND
 - Field: `Leave_Days__c` less than or equal to `15`
- Select **"Automatically assign to approver(s)"**

Click **Next**.

Step 4: Specify Entry Criteria Again (Confirmation)

You may be prompted to reconfirm or redefine. Use the same logic:

vbnet

```
Status__c EQUALS Submitted
AND
Leave_Days__c LESS THAN OR EQUAL TO 15
```

Click **Next**.

Step 5: Select Approver Field

Choose "Automatically assign using a standard or custom hierarchy field"

- Use the `Manager__c` field

This will be **Step 1 Approver: Manager**

Click **Next**.

Step 6: Specify Email Template

(Optional) – Select or create an email template to notify the manager upon submission.

Click **Next**.

Step 7: Final Approval, Rejection, and Recall Actions

Final Approval Actions:

- Field Update → Set `Status__c` to "Approved"
- (Optional) Send email to employee

Final Rejection Actions:

- Field Update → Set **Status__c** to "Rejected"
- (Optional) Send email to employee

Recall Actions:

- Field Update → Set **Status__c** to "Draft"

Click **Next**.

Step 8: Activate the Approval Process

Click **Activate** to enable it.

Adding Multiple Steps (HR Step)

Now we need to add **Step 2** for HR:

1. After activation, click **"Add New Step"**
 2. Name: **HR Approval**
 3. Criteria: "All records enter this step"
 4. Select **Approver**: Either a **Queue** called "HR Team" or a **specific user**
 5. Actions:
 - Field Update: **Status__c** = "Pending HR Approval"
-

Testing the Approval Process

1. Create a new Leave Request Record
 - Set **Status__c** to "Submitted"
 - **Leave_Days__c** = 10
 2. Save and click **"Submit for Approval"**
 3. Manager receives an email and approval request
 4. After approval, HR receives a request
 5. Final status is updated automatically based on approval or rejection
-

Bonus: Add Submit Button to Page Layout

Ensure you have a **"Submit for Approval"** button:

1. Go to **Object Manager** → **Leave_Request__c** → **Page Layouts**

2. Edit the layout
3. Add the **"Submit for Approval"** button to the layout

Would you like me to provide a diagram or XML export of the approval process, or help with Apex automation for submission?



Apex automation for submission

Great! Let's walk through **how to submit a record for approval programmatically using Apex** in Salesforce.

✓ Use Case

You want to **automatically submit a Leave Request record** into the approval process via Apex, instead of requiring the user to click the **"Submit for Approval"** button manually.

🔧 Steps to Automate Submission via Apex

🎯 1. Preconditions

Before calling the Apex logic, make sure:

- Approval Process is **active**.
- Entry criteria are met (e.g., `Status__c = 'Submitted'` and `Leave_Days__c <= 15`).
- The record must be **inserted** or already exist in the database.

✏️ 2. Sample Apex Code: Auto-Submit for Approval

```
apex

Id leaveRequestId = 'a012h00001XyZAb'; // Replace with real ID or use dynamically

Leave_Request__c leaveRequest = [SELECT Id, Status__c FROM Leave_Request__c WHERE Id = :leaveRequestId];

// Check if record meets your entry criteria
if (leaveRequest.Status__c == 'Submitted') {

    // Create the approval request object
    Approval.ProcessSubmitRequest req = new Approval.ProcessSubmitRequest();
    req.setComments('Automatically submitted via Apex');
    req.setObjectId(leaveRequest.Id);

    // Optionally, skip the first step and specify a next approver
    // req.setNextApproverIds(new Id[] {UserInfo.getUserId()});

    // Submit the request
    Approval.ProcessResult result = Approval.process(req);
}
```

```
// Check result
if (result.isSuccess()) {
    System.debug('Approval Request submitted successfully. Instance Id: ' +
result.getInstanceId());
} else {
    System.debug('Failed to submit for approval: ' + result.getErrors());
}
}
```

⚠ 3. Important Notes

Feature	Notes
Approval.ProcessSubmitRequest	Used to submit a record for approval
setObjectId()	Required — ID of the record to submit
setNextApproverIds()	Optional — manually assign the next approver
Approval.process()	Executes the submission request
Record Locking	The record will be locked after submission (if configured)
Error Handling	Always check for result.isSuccess() and .getErrors()

🔧 4. Use It in Trigger (Optional)

You can place this inside a trigger to automatically submit upon insert/update:

```
apex

trigger AutoSubmitLeaveRequest on Leave_Request__c (after insert, after update) {
    for (Leave_Request__c lr : Trigger.new) {
        if (lr.Status__c == 'Submitted') {
            try {
                Approval.ProcessSubmitRequest req = new Approval.ProcessSubmitRequest();
                req.setComments('Auto-submitted by trigger');
                req.setObjectId(lr.Id);
                Approval.ProcessResult result = Approval.process(req);
                System.debug('Approval submitted for ID: ' + lr.Id);
            } catch (Exception e) {
                System.debug('Error submitting for approval: ' + e.getMessage());
            }
        }
    }
}
```

🔧 5. Best Practices

- Avoid recursive trigger submission — use a static variable guard.
- Ensure the approval process is properly configured and activated.
- Always test in sandbox first.

✅ Summary

Using **Approval.ProcessSubmitRequest**, you can:

- Submit any object record for approval automatically.
- Customize approvers.
- Check for success/failure via result object.

Would you like to automatically approve/reject a step via Apex too (e.g., manager approves without logging in)?



Yes

Perfect! Let's now go one step further and learn how to **approve or reject an approval step using Apex**, without requiring the approver to manually take action in the UI.

✓ Use Case

You want to **automatically approve or reject** a step in an approval process using Apex, for example:

- A manager or system auto-approves if certain conditions are met.
- A rejection happens due to a rule (e.g., too many leave days).

Classes Used

To programmatically approve/reject a record:

- Use **Approval.ProcessWorkitemRequest** – to **approve/reject** a step
- Get the **WorkItem Id** of the current approval step using SOQL

Step-by-Step Apex Code

1. Auto-Approve a Record

```
apex

// Get the Approval Process Instance (the submission)
List<ProcessInstance> piList = [SELECT Id, TargetObjectId, Status
                                FROM ProcessInstance
                                WHERE TargetObjectId = :leaveRequestId
                                AND Status = 'Pending'];

for (ProcessInstance pi : piList) {

    // Get the WorkItems (approval steps) of that instance
    List<ProcessInstanceWorkitem> workItems = [
        SELECT Id, ActorId, ProcessInstanceId
        FROM ProcessInstanceWorkitem
        WHERE ProcessInstanceId = :pi.Id
    ];
```

```
for (ProcessInstanceWorkitem item : workItems) {
    // Create a request to approve
    Approval.ProcessWorkitemRequest req = new Approval.ProcessWorkitemRequest();
    req.setComments('Auto-approved via Apex');
    req.setAction('Approve'); // Or 'Reject'
    req.setWorkitemId(item.Id);

    // Perform approval
    Approval.ProcessResult result = Approval.process(req);

    if (result.isSuccess()) {
        System.debug('Record Approved. WorkItem ID: ' + item.Id);
    } else {
        System.debug('Failed to approve: ' + result.getErrors());
    }
}
}
```

✗ 2. Auto-Reject a Record

Change `setAction('Approve')` to `'Reject'`:

```
apex

req.setAction('Reject');
req.setComments('Rejected automatically because leave days exceeded limit.');
```

Understanding ProcessInstanceWorkitem

Field	Meaning
Id	ID of the approval step (needed for action)
ActorId	Approver's User ID
ProcessInstanceId	Link to the approval process instance

Common Pitfalls

Issue	Fix
No work items found	Make sure the record is actually pending approval.
Invalid ID error	Ensure you're using a valid WorkItem ID from ProcessInstanceWorkitem .
Record not locked/unlocked	Use approval actions to manage that during config.

Optional: Approve as Specific User

To approve as a different user (e.g., manager), you must run the code **in that user's context** (e.g., via `System.runAs()` in a test class). In real logic, only the assigned approver can approve unless delegated.

Example Use Case

Automatically approve a Leave Request if leave days ≤ 5 :

```
apex

if (leaveRequest.Leave_Days__c <= 5) {
    // Fetch ProcessInstance and approve as above
}
```

✓ Summary

Task	Apex Class Used
Submit for Approval	Approval.ProcessSubmitRequest
Approve/Reject a Step	Approval.ProcessWorkitemRequest
Fetch Approval Step	ProcessInstanceWorkitem

Would you like me to help wrap this logic into a **utility class** or **trigger handler pattern** for reuse across multiple objects or flows?



Difference between manual approval process, flow approval process and apex approval process in tabular format

Here's a detailed **tabular comparison** of the three types of approval submission methods in Salesforce:

✓ Manual vs Flow vs Apex Approval Process

Feature / Criteria	Manual Approval Process	Flow-Based Approval Process	Apex-Based Approval Process
Submission Method	User clicks "Submit for Approval" on the record page	Flow auto-submits based on logic and criteria	Apex code submits record using Approval class
Automation Level	✗ Manual only	✓ No-code automation	✓ Full code automation
User Involvement	Required	Not required	Not required
Customization Flexibility	Limited to standard UI actions	Medium (Flow logic)	High (any condition or business rule in code)
Dynamic Approver Assignment	Possible via Approval Process Setup	Possible via Flow logic + variables	Fully dynamic via logic and queries
Trigger on Create/Update	✗ No	✓ Yes (Record-Triggered Flow)	✓ Yes (in Triggers or Apex classes)
Error Handling	Minimal (only UI messages)	Moderate (via Flow Fault Paths)	Robust (try-catch, custom logs, etc.)
Code Knowledge Required	✗ No	⚠ Basic Flow knowledge	✓ Yes – Apex knowledge needed
Ease of Maintenance	Easy	Moderate	Harder – requires

Feature / Criteria	Manual Approval Process	Flow-Based Approval Process	Apex-Based Approval Process
			developer support
Record Locking	✅ Yes (configured in Approval Setup)	✅ Yes (inherits from approval process config)	✅ Yes (same as above)
Recall/Resubmit Support	✅ Yes (via UI)	✅ Yes (via Flow)	✅ Yes (via Apex logic)
Batch/Bulk Submission Support	❌ No	⚠️ With limitations	✅ Yes (via collections in Apex)
Use Case Suitability	Simple, user-driven approvals	Declarative automation needs	Complex logic or integrations (e.g., auto-approval rules)
Best for	Admins and end-users	Admins/Declarative Developers	Developers with complex business rules

When to Use What?

Situation	Recommended Method
Simple submission by user	Manual
Auto-submit on record creation	Flow
Conditional submit + auto approve/reject	Apex
Bulk approval requests from integration	Apex
Approver determined via formula field	Any method (all support this)
Approval chaining based on logic	Apex or Flow (complex)

Let me know if you'd like a **diagram**, **code-template**, or **flow-builder steps** for any of these approaches!



Code template and flow builder steps for these

Sure! Let's break this into two parts:

PART 1: Apex Code Template for Approval Submission and Auto-Approval

We'll build a **reusable Apex utility class** to:

1. Submit a record for approval
2. Optionally auto-approve or auto-reject based on logic

Apex Utility Class: ApprovalUtils

apex

```
public class ApprovalUtils {

    // Submit record for approval
    public static void submitForApproval(Id recordId, String objectName) {
        try {
            // Submit for Approval
            Approval.ProcessSubmitRequest req = new Approval.ProcessSubmitRequest();
            req.setObjectId(recordId);
            req.setComments('Auto-submitted via Apex');

            Approval.ProcessResult result = Approval.process(req);

            if (result.isSuccess()) {
                System.debug('Submitted for approval: ' + recordId);
            } else {
                System.debug('Submission failed: ' + result.getErrors());
            }
        } catch (Exception e) {
            System.debug('Error submitting for approval: ' + e.getMessage());
        }
    }

    // Auto-approve the current approval step
    public static void autoApproveCurrentStep(Id recordId) {
        List<ProcessInstance> instances = [SELECT Id FROM ProcessInstance
                                         WHERE TargetObjectId = :recordId AND Status =
'Pending'];

        for (ProcessInstance pi : instances) {
            List<ProcessInstanceWorkitem> items = [SELECT Id FROM ProcessInstanceWorkitem
                                                  WHERE ProcessInstanceId = :pi.Id];
            for (ProcessInstanceWorkitem item : items) {
                Approval.ProcessWorkitemRequest req = new Approval.ProcessWorkitemRequest();
                req.setWorkitemId(item.Id);
                req.setAction('Approve');
                req.setComments('Auto-approved by Apex based on logic');

                Approval.ProcessResult result = Approval.process(req);

                if (result.isSuccess()) {
                    System.debug('Auto-approved work item: ' + item.Id);
                } else {
                    System.debug('Auto-approval failed: ' + result.getErrors());
                }
            }
        }
    }
}
```

✓ How to Use in Trigger or Class

apex

```
// Example use in a trigger or class
Leave_Request__c req = [SELECT Id, Leave_Days__c FROM Leave_Request__c WHERE Id = :someId];
if (req.Leave_Days__c <= 5) {
    ApprovalUtils.submitForApproval(req.Id, 'Leave_Request__c');
    ApprovalUtils.autoApproveCurrentStep(req.Id);
}
```



PART 2: Flow Builder Steps for Approval Process

You can also automate this with **Record-Triggered Flows**.

Use Case: Auto-Submit a Record for Approval in Flow

Prerequisites

- Active approval process on object
- "Submit for Approval" button is available in UI
- Entry criteria set

Flow Builder Steps (Submit for Approval)

1. Go to Setup → Flows → New Flow
 - Choose **Record-Triggered Flow**
2. Configure Trigger
 - Object: `Leave_Request__c`
 - Trigger When: **A record is created or updated**
 - Entry Condition:
 - `Status__c Equals Submitted`
 - `Leave_Days__c <= 15`
 - Optimize for: **Actions and Related Records**
3. Add Action → Submit for Approval
 - Click + → Select **Action**
 - Action Type: **Submit for Approval**
 - Input Values:
 - **Record ID:** `{Record.Id}`
 - **Comments:** "Submitted via Flow"
4. Save and Activate Flow

Optional: Add Auto-Approval Step via Flow

Unfortunately, **auto-approving a step** is **not natively supported in Flow**, but you can:

- Create a **scheduled path** or **invocable Apex** to auto-approve.
- Use **Apex Action** in Flow to call `ApprovalUtils.autoApproveCurrentStep()` as a custom Apex action.

Summary

Feature	Flow	Apex
Submit for approval	Action: Submit for Approval	<code>Approval.ProcessSubmitRequest()</code>
Auto-approve step	Use Invocable Apex	<code>Approval.ProcessWorkitemRequest()</code>

Feature	Flow	Apex
Use in trigger/automation	Flow Trigger	Apex Trigger/Class
Custom logic control	Limited	Full control

Would you like me to create and share the **Apex Invocable method** so that it can be **called inside Flow as an action** to auto-approve?



escalation rule

✓ What is an Escalation Rule in Salesforce?

An **Escalation Rule** in Salesforce automatically **reassigns cases** or sends alerts **based on specific criteria and time-based conditions**.

Escalation rules are mainly used with **Cases** in **Service Cloud** to ensure **timely response and resolution** of customer issues.

Purpose of Escalation Rules

- Ensure **cases don't go unanswered** beyond a deadline.
- Automatically **escalate** (reassign or notify) a support rep/manager if a case isn't responded to within a specified time.
- Improve **SLA (Service Level Agreement)** compliance.

Key Concepts

Term	Description
Escalation Rule	A rule that contains multiple entries, each with criteria and time triggers
Rule Entry	Defines criteria and what to do when the rule matches
Escalation Action	What happens (e.g., assign user, notify via email, etc.)
Business Hours	Used to calculate escalation timing
SLA	Escalation rules help enforce SLAs by ensuring timely case handling

Components of an Escalation Rule

1. **Rule Name**
2. **Rule Entries** (multiple per rule)
 - Sort Order
 - Criteria (e.g., Priority = High)

3. Escalation Actions

- Time Criteria (e.g., 4 hours after creation)
- Assign to User/Queue
- Email Notification (optional)

4. Business Hours and Case Milestones

How to Configure Escalation Rules (Step-by-Step)

Step 1: Enable Escalation Rules

- Go to **Setup** → **Escalation Rules**
- Click **New**
 - Rule Name: **High Priority Escalation**
 - Active: ☒ Yes
 - Save

Step 2: Add Rule Entries

- After creating the rule, click **Rule Name**
- Click **New Rule Entry**
 - Sort Order: 1
 - Criteria:
 - Case Priority equals High
 - Save

Step 3: Add Escalation Actions

- On the rule entry, click **New Escalation Action**
 - **Age Over:** 4 hours (when to escalate)
 - **Assign to:** Queue **Escalated Cases** or a specific User
 - (Optional) Select **Email Notification Template**
 - (Optional) Add additional escalation levels (e.g., 8 hours to escalate to manager)

Step 4: Set Business Hours (Optional but recommended)

- Go to **Setup** → **Business Hours**
- Define business hours (e.g., 9 AM – 6 PM, Mon–Fri)
- Link business hours to escalation rules for accurate timing

Example: Case Escalation Rule

Criteria	Action
Case.Priority = High	After 4 hours, assign to Support Manager
Case.Status ≠ Closed	After 8 hours, assign to Director of Support + send email

Difference Between Escalation Rule and Approval Process

Feature	Escalation Rule	Approval Process
Object	Only for Cases	Any object with approval support
Purpose	Ensure timely response, reassign cases	Route records for approval
Trigger	Time + criteria based	Manual or automated
Actions	Reassign, send alerts	Approve, reject, update, lock record
Multiple Steps	Multiple time-based actions	Multi-step with approvers

Best Practices

- Always define **business hours** for SLA accuracy.
- Create separate rules for **low, medium, high priority**.
- Send escalation **emails to managers** or **queues**, not just individuals.
- Monitor escalations using **Reports** or **Case History**.

Would you like a **real-world example configuration**, a **use-case diagram**, or a **sample email template** used during escalation?



task,case,lead,opportunity,contacts,account,campaigns

It looks like you've listed several **standard Salesforce objects**:

→ **Task, Case, Lead, Opportunity, Contact, Account, and Campaign.**

Below is a **tabular breakdown** of each object, including **purpose, key fields, relationships, and usage examples**:

Salesforce Standard Objects Overview

Object	Purpose	Key Fields	Related To	Example Use Case
Task	To log activities, to-dos , or follow-ups (assigned to users).	Subject, Status, Priority, Due Date, Assigned To	Related to: Lead, Contact, Account, Opportunity, Case, etc.	Reminder to follow-up with a lead next week

Object	Purpose	Key Fields	Related To	Example Use Case
Case	Used for customer support and issue tracking.	Subject, Description, Status, Priority, Case Origin	Related to: Account, Contact, User, Solution	Logging a support ticket from a customer
Lead	Potential customer not yet qualified.	Name, Company, Status, Lead Source	Converts into: Account, Contact, Opportunity	New inquiry from a website form
Opportunity	Tracks sales deals and their progress.	Name, Stage, Amount, Close Date	Related to: Account, Contact, Campaign	Selling software license worth \$5,000
Contact	A person associated with an Account (Customer or Prospect).	Name, Email, Phone, Title	Related to: Account, Opportunities, Campaigns, Cases	Jane Doe works at Acme Corp as a VP of Sales
Account	Represents a company or organization (B2B), or household (B2C).	Name, Industry, Type, Website	Related to: Contacts, Opportunities, Cases, Campaigns	Acme Corp is a corporate customer
Campaign	Marketing initiative used to track outreach and engagement.	Name, Type, Status, Start/End Dates, Budgeted Cost	Related to: Leads, Contacts, Opportunities	Email campaign for summer product launch

Object Relationships in Salesforce

plaintext

```
Lead → [Convert] → Account + Contact + Opportunity
Account ↔ Contacts (1:N)
Account ↔ Opportunities (1:N)
Opportunity ↔ Contact Roles (N:N)
Contact ↔ Campaigns via Campaign Members (N:N)
Task/Activity ↔ Related to any of the above (Polymorphic)
Case ↔ Account or Contact (1:1)
```

Use Case Example: Lead to Opportunity

Step	Object	Action
1	Lead	Capture new user from form
2	Task	Create a follow-up task
3	Lead (Convert)	Lead converts → creates Account + Contact + Opportunity
4	Opportunity	Sales rep updates opportunity stage
5	Account	Stores customer info; links to multiple contacts and opportunities
6	Campaign	Link contacts/leads to campaigns for marketing ROI tracking
7	Case	Customer raises a support request → case linked to account and contact

Summary Table of Object Usage

Object	Used In	Typical User Roles
Task	Sales, Support, Marketing	Sales Reps, Agents, Field Reps
Case	Customer Support	Support Agents, Service Managers
Lead	Pre-sales, Marketing	SDRs, Marketing Team
Opportunity	Sales Funnel/CRM	Account Executives, Sales Managers
Contact	Customer Data	Everyone – especially Sales & Support
Account	Business Entity Info	Sales Reps, Support Agents, Finance
Campaign	Marketing Performance Tracking	Marketing Managers, Campaign Planners

Would you like a **diagram of how these objects are connected**, or a **real-world CRM flow example** from Lead to Case support?



give me all standards objects relationship

Here's a comprehensive overview of the **standard object relationships in Salesforce**, particularly focusing on the **core CRM objects**. These include:

- **Lead**
- **Account**
- **Contact**
- **Opportunity**
- **Case**
- **Campaign**
- **Task/Event (Activities)**
- **User**

Standard Object Relationships in Salesforce

Lead Relationships

Field/Relation	Related Object	Description
ConvertedAccountId	Account	Account created after conversion
ConvertedContactId	Contact	Contact created after conversion
ConvertedOpportunityId	Opportunity	Opportunity created after conversion (optional)
OwnerId	User	Owner of the Lead
CampaignId	Campaign	Campaign the lead is a member of

Account Relationships

Field/Relation	Related Object	Description
Contacts	Contact (1:N)	An account can have multiple contacts
Opportunities	Opportunity (1:N)	An account can have many opportunities
Cases	Case (1:N)	An account can have many cases
Parent Account	Account (self)	For hierarchy of accounts (e.g., parent company)
Campaign Influence	Campaigns (N:N)	Campaigns influencing related opportunities
Tasks/Events	Task/Event (1:N)	Activities related to this account

● Contact Relationships

Field/Relation	Related Object	Description
AccountId	Account	The company the contact is associated with
Opportunities	Opportunity	Via Contact Roles (many-to-many)
Campaigns	Campaign	Via Campaign Member object (many-to-many)
Cases	Case	Contacts can be linked to support cases
Tasks/Events	Task/Event	Related activities

● Opportunity Relationships

Field/Relation	Related Object	Description
AccountId	Account	Opportunity belongs to an account
Contact Roles	Contact (N:N)	Multiple contacts involved in the deal
Campaign Influence	Campaign	Which campaigns influenced the opportunity
Tasks/Events	Task/Event	Related activities like meetings, follow-ups

● Case Relationships

Field/Relation	Related Object	Description
AccountId	Account	Account that raised the case
ContactId	Contact	Contact person for the case
OwnerId	User or Queue	Support agent or queue assigned
ParentCaseId	Case (self)	Parent case (for child/split case logic)
Tasks/Events	Task/Event	Activities on the case

● Campaign Relationships

Field/Relation	Related Object	Description
Campaign Members	Lead, Contact	Tracks who was targeted by the campaign
Opportunities	Opportunity	Influenced Opportunities (indirect via Influence)
Parent Campaign	Campaign (self)	Hierarchical grouping of campaigns

Task and Event (Activities)

Field/Relation	Related Object	Description
WhoId	Contact or Lead	Person the activity is related to (polymorphic)
WhatId	Account, Opp, etc.	Record the activity is about (polymorphic)
OwnerId	User	Who owns the activity

User Relationships

Related To	Description
Leads, Accounts, etc.	Users are owners of most records
Case/Opportunity Team	Can be members of teams assigned to Cases/Opps
Queues	Users can be members of queues

Summary Diagram (Text Format)

vbnet

```

Lead
├─ (Converts to) → Account, Contact, Opportunity
│   ├── Contact ↔ Campaign (via Campaign Member)
│   ├── Contact ↔ Opportunity (via Contact Role)
│   ├── Account ↔ Opportunity (direct link)
│   ├── Account ↔ Case
│   ├── Account ↔ Task/Event
│   ├── Opportunity ↔ Campaign (Campaign Influence)
│   └─ All Objects ↔ Task/Event (Activities)

```

Best Practice Tips

- Use **lookup fields** to model 1:N relationships.
- Use **junction objects** like **Campaign Member** and **Contact Role** for N:N relationships.
- Use **Activities (Task/Event)** to track interaction history across almost all key objects.
- Enable **Parent Account** and **Parent Campaign** for hierarchy visualizations.

Would you like a **visual ERD (Entity Relationship Diagram)** image of these objects and their relationships?



use of user object

The **User** object in Salesforce represents **individual users** of the Salesforce platform. It's one of the most important standard objects because it controls **authentication, permissions, ownership, and assignments** across the entire system.

✓ Primary Uses of the User Object

Purpose	Description
Authentication	Represents each logged-in person — their login credentials, profile, etc.
Record Ownership	Tracks who owns records (e.g., Account.OwnerId, Case.OwnerId, etc.)
Role and Profile Assignment	Determines what data and features a user can access
Sharing and Visibility	Drives record-level access in sharing rules and role hierarchies
Approvals and Workflow	Used in Approval Processes, Task Assignments, and Email Alerts
Audit and Tracking	Tracks who created or last modified a record (CreatedById, LastModifiedById)
Login and Access Logs	Used in login history, session management, IP restrictions, etc.
Queue and Team Membership	Users can belong to Queues, Public Groups, Opportunity Teams, Case Teams
Email Communication	Used to send notifications, assign tasks, and route cases

Key Fields in the User Object

Field API Name	Description
Id	Unique User ID
Username	Login email/username (must be unique across orgs)
Email	User's email address
ProfileId	ID of assigned Profile (controls permissions)
RoleId	ID of assigned Role (used for role hierarchy sharing)
IsActive	Whether the user account is active
Alias	Short name or nickname
Name	User's full name
UserType	'Standard', 'Partner', 'Customer Portal', etc.
TimeZoneSidKey	User's time zone
LocaleSidKey	User's locale
LanguageLocaleKey	Language preferences

Relationships with Other Objects

Object	How User is Related
Account, Case, Lead, Opportunity, etc.	OwnerId — shows who owns the record
Task/Event	OwnerId — who is responsible for the activity
Approval Process	Users can be assigned as approvers
Queues	Users can be members
Chatter	Users can post, comment, and receive mentions
Custom Objects	Can have lookup fields pointing to User (e.g., Assigned_To__c)

Common Use Cases

1. Assigning Ownership

plaintext

```
Account.OwnerId → User
Case.OwnerId → Support Agent (User)
Opportunity.OwnerId → Sales Rep (User)
```

2. Automation & Workflows

- Notify `User.Email`
- Assign approval to `User.Id`
- Set `Assigned To` in Tasks

3. Visibility and Sharing

- **Role hierarchy:** Users higher in the hierarchy can view subordinates' data.
- **Manual sharing:** Share record with a specific `User`.

4. Audit Tracking

- `CreatedById` and `LastModifiedById` fields reference `User` object.

Example in SOQL

sql

```
SELECT Id, Name, Email, Profile.Name, Role.Name
FROM User
WHERE IsActive = true
```

Limitations

Limitation	Description
Username must be unique globally	You can't reuse an existing username in another Salesforce org.
Max active users per license	You are limited by the number of purchased licenses
Can't delete users	Users can only be deactivated , not deleted

Summary

Feature	User Object Role
Record Ownership	Owns leads, opportunities, cases, etc.

Feature	User Object Role
Permissions	Defined by Profile and Role
Workflow/Approval	Can be assigned for approvals, tasks, email notifications
Collaboration	Chatter posts, followers, @mentions
System Audit	Tracks who created/updated records

Would you like to see a **real-life user record layout**, **automated email alert using user email**, or **trigger example assigning a user dynamically**?



trigger example assigning a user dynamically

Here is a **trigger example in Salesforce Apex** that demonstrates how to **assign a specific user dynamically** to a record (e.g., a **Case**, **Lead**, or **Custom__c** object), based on custom logic (like region, type, priority, etc.).

✓ Use Case

When a new **Case** is created, assign the owner to a specific **User** based on the Case's **Priority** field:

- If **Priority** = **High**, assign to **High Priority Agent**
- If **Priority** = **Medium**, assign to **Medium Priority Agent**
- Else, assign to **Default Support User**

🧱 Preparation

1. Find User Ids for the users to whom cases will be assigned.
2. Ideally store them in **Custom Metadata**, but for simplicity, this code hardcodes the Ids (not recommended for production).

⚙️ Apex Trigger Code

```
apex

trigger AssignCaseOwner on Case (before insert) {
    // Fetch users dynamically by email or alias (recommended)
    Map<String, Id> priorityToUserMap = new Map<String, Id>();

    // Query Users (Assuming aliases or emails help identify them)
    for (User u : [
        SELECT Id, Alias, Email FROM User
        WHERE Alias IN ('highuser', 'mediumuser', 'default')
        AND IsActive = true
    ]) {
        if (u.Alias == 'highuser') {
```

```

        priorityToUserMap.put('High', u.Id);
    } else if (u.Alias == 'mediumuser') {
        priorityToUserMap.put('Medium', u.Id);
    } else if (u.Alias == 'default') {
        priorityToUserMap.put('Default', u.Id);
    }
}

for (Case c : Trigger.new) {
    if (c.Priority == 'High' && priorityToUserMap.containsKey('High')) {
        c.OwnerId = priorityToUserMap.get('High');
    } else if (c.Priority == 'Medium' && priorityToUserMap.containsKey('Medium')) {
        c.OwnerId = priorityToUserMap.get('Medium');
    } else if (priorityToUserMap.containsKey('Default')) {
        c.OwnerId = priorityToUserMap.get('Default');
    }
}
}

```

Test Class

```

apex

@isTest
public class AssignCaseOwnerTest {

    @isTest
    static void testHighPriorityAssignment() {
        // Setup Users
        User highUser = [SELECT Id FROM User WHERE Alias = 'highuser' LIMIT 1];

        // Create test Case
        Case c = new Case(
            Subject = 'Test High Priority',
            Priority = 'High',
            Status = 'New'
        );

        insert c;

        // Re-fetch and verify owner
        Case insertedCase = [SELECT OwnerId FROM Case WHERE Id = :c.Id];
        System.assertEquals(highUser.Id, insertedCase.OwnerId);
    }
}

```

Best Practices

Practice	Reason
 Use Custom Metadata	Avoid hardcoding User Ids in Apex
 Always Write Tests	Cover different scenarios (High, Medium, Default)
 Handle Missing Users	Add fallback logic in case users aren't found
 Avoid Hardcoded Ids	Breaks in different orgs or sandboxes
 Use Field-Level Security	Ensure owner field can be set by the running user

Related Use Cases

- Assigning Leads to users based on Region
- Assigning Opportunities to Account Managers
- Assigning Cases to Queues (just set `OwnerId = Queue.Id`)

Would you like an example of assigning **to a Queue**, using **Custom Metadata**, or implementing this logic using **Flows instead of Apex**?



role of user in experience cloud

In **Salesforce Experience Cloud**, the **User object** plays a **crucial role** in managing access, personalization, and collaboration for external users (like customers, partners, or employees). Experience Cloud (formerly **Community Cloud**) allows you to create branded portals, forums, and sites — and **Users** are the key to who can log in, what they can see, and what they can do.

✓ Roles of a User in Experience Cloud

Role	Explanation
Login Access	A User (Customer, Partner, or Employee) must exist and be active to access an Experience Site.
Profile & Permission Set	Controls what the user can access within the community: objects, fields, tabs, components, and pages.
Role (Hierarchy)	For Partner and Customer users, roles define access to records (used in sharing rules). Example: Partner users can see their subordinates' records.
Account Association	External users (Partner/Customer) are linked to an Account — which ties into data visibility and sharing.
Portal User Type	Determines the behavior and capabilities of the user: Customer Community , Partner Community , or Employee Community .
Chatter & Collaboration	Users can post, comment, follow topics, and collaborate with others (based on permissions).
Personalization	Experience Cloud sites can display dynamic content based on the logged-in user (geography, language, preferences, etc.).
Record Ownership	Users can own records like Cases, custom object records, etc., and trigger sharing rules or workflows.
Branding & Experience	Logged-in user's data can be used to personalize page content using audience targeting or visibility filters.

Types of Experience Cloud Users

User Type	Description
Customer Community User	Used for end customers. Can access self-service portals, cases, knowledge base, etc.
Customer Community Plus	Adds reports, dashboards, and sharing access.

User Type	Description
User	
Partner Community User	Used for partners (resellers, distributors). Adds role hierarchy, lead/opportunity access, etc.
Employee Community User	For internal employees using the community site (usually with a limited license).

User Relationships in Experience Cloud

Object	Relationship with User
Account	External users must be associated with an Account (as Contact → User)
Contact	Experience users are created from Contacts
Role	Partner/Customer Plus Users get auto-created roles
Permission Set / Profile	Controls object/tab/page access
Experience Site Membership	Defines which users can access which sites

Creating a Community User (Typical Flow)

1. Create an **Account**
2. Create a **Contact** under that Account
3. Enable the Contact as a **User**
 - Choose a profile like **Customer Community User**
4. Assign proper **role**, **profile**, and optionally **permission sets**
5. Add the User to the **Experience Site Membership**

User Access Control in Experience Cloud

Control Layer	Description
Profile	Basic object/field-level access
Permission Sets	Extend permissions without changing profile
Role Hierarchy	Enables sharing records up the hierarchy
Sharing Rules	Share records with users based on roles or groups
Login IP Restrictions	Secure external user access
Experience Site Membership	User must be added to site to access it

Example Scenario

You have a customer portal. When a customer signs up:

- A **Contact** is created under the **Account** "ABC Corp."

- A **User** is created from that Contact with the **Customer Community Profile**
- The user logs into the Experience Site and:
 - Views Knowledge Articles
 - Submits Cases
 - Tracks Case Status
 - Receives email alerts and Chatter notifications

✓ Summary: User Role in Experience Cloud

Function	Description
Authentication	Who can log into the Experience Site
Authorization	What data and features the user can access
Personalization	Customize UI based on user's role, location, or preferences
Ownership	Records (like Cases, Custom Objects) created or owned by the user
Collaboration	Enable Chatter and community interactions

Would you like an **end-to-end flow for enabling a Contact as a Portal User**, or a **Flow Builder example to auto-create an Experience Cloud User**?



Flow Builder example to auto-create an Experience Cloud User

Here is a **complete Flow Builder example to automatically create an Experience Cloud User** (portal user) in Salesforce when a new **Contact** is created.

Objective

When a new **Contact** is created under a specific **Account** (e.g., a Customer Account), Salesforce should **auto-create an Experience Cloud User** with the selected Profile and Role, and add them to the Experience Site.

✓ Pre-requisites

1.  The Experience Site (Community) is already created and **active**.
2.  The **Profile** to assign (e.g., **Customer Community User**) is available.
3.  The **Permission Set License** (**Customer Community**, **Partner Community**, etc.) is active.
4.  The **Account** is enabled as a **Partner Account** or **Customer Account** (if required).
5.  The Experience Site must be **configured for self-registration or manually accessible**.

Steps in Flow Builder

► Type: Record-Triggered Flow

- **Trigger Object:** Contact
- **Trigger:** When a **record is created**
- **Entry Conditions:**
 - AccountId is not null
 - Email is not null
- **Run As:** System Context (with sharing)

Step 1: Get the Experience Cloud Site

We'll retrieve the Site where the user will be created.

- **Element:** Get Records
- **Object:** Site
- **Condition:** Name = 'Your_Experience_Cloud_Site_Name'
- **Store First Record Only**

Step 2: Get the Portal User Profile

- **Element:** Get Records
- **Object:** Profile
- **Condition:** Name = 'Customer Community User'
- **Store First Record Only**

Step 3: Create User Record

- **Element:** Create Records
- **Object:** User
- **Field Values** (example):

Field	Value
FirstName	Contact.FirstName
LastName	Contact.LastName
Username	Contact.Email + '.portal'
Email	Contact.Email
Alias	LEFT(Contact.LastName, 8)
TimeZoneSidKey	America/Los_Angeles (or your org's default)

Field	Value
LocaleSidKey	en_US
EmailEncodingKey	UTF-8
LanguageLocaleKey	en_US
ContactId	Contact.Id
ProfileId	From the Profile record you fetched earlier
UserRoleId	Optional: fetch or assign a role
Username (Unique)	Ensure uniqueness, or use GUID if needed

⚠ Important: You must also assign the proper **Community License** (Permission Set License) to the user via code or manually.

🔧 Step 4: Add User to Site Membership (Optional)

If needed, use an **Apex Action** or a **custom invocable Apex class** to assign the user to the site. Salesforce does not directly expose the **NetworkMemberGroup** object in Flow.

Example: Use an Apex Action

```
apex

public with sharing class AddUserToSite {
    @InvocableMethod
    public static void addUsersToSite(List<InputWrapper> requests) {
        List<NetworkMemberGroup> memberships = new List<NetworkMemberGroup>();
        for (InputWrapper req : requests) {
            memberships.add(new NetworkMemberGroup(
                NetworkId = req.siteId,
                MemberId = req.userId,
                GroupType = 'User'
            ));
        }
        insert memberships;
    }

    public class InputWrapper {
        @InvocableVariable public Id siteId;
        @InvocableVariable public Id userId;
    }
}
```

Then in Flow:

- Add Apex Action: **AddUserToSite**
- Inputs:
 - **siteId** = Site.Id
 - **userId** = CreatedUser.Id

✅ Summary Flow Structure

Element Type	Description
Trigger	When Contact is created
Get Records	Get Site
Get Records	Get Profile
Create Records	Create User
Apex Action	Add User to Site (optional)

Notes

- Username must be unique across all Salesforce orgs.
- Only contacts under enabled accounts (as partner/customer) can be converted to users.
- If you face errors like `"INSUFFICIENT_ACCESS_ON_CROSS_REFERENCE_ENTITY"`, ensure the Account is enabled as a partner/customer account.

Would you like me to **package this Flow in JSON** format for deployment, or provide a **custom Lightning Web Component** for self-registration as well?



difference between whatID and whoID

In Salesforce, **WhatId** and **WhoId** are fields found on **Activity records** like **Tasks** and **Events**. These fields are used to relate the activity to other Salesforce records — but each serves a different purpose.

Difference Between WhatId and WhoId

Aspect	WhatId	WhoId
Meaning	Refers to objects (things)	Refers to people
Related To	Any object that is not a person	Leads or Contacts
Examples	Opportunity, Account, Campaign, Custom Object	Contact, Lead
Field Type	Polymorphic lookup (many-to-one)	Polymorphic lookup
Field API Name	WhatId	WhoId
Used In	Task and Event objects	Task and Event objects
Multiple Records?	No (1 record)	No (1 record)

Use Case Examples

Example 1: Task on an Opportunity

plaintext

Subject: Follow-up call
WhoId: Contact John Doe
WhatId: Opportunity "ABC Deal"

- **WhoId:** The person you're calling (John Doe)
- **WhatId:** The deal you're calling about (ABC Deal)

Example 2: Event Linked to a Case

plaintext

Subject: Meeting to discuss issue
WhoId: Lead Jane Smith
WhatId: Case "0001234 - Login Problem"

- **WhoId:** The person you're meeting (Jane Smith)
- **WhatId:** The Case (the issue being discussed)

Tips

Scenario	Use WhoId?	Use WhatId?
Linking to a Contact or Lead	✓ Yes	✗ No
Linking to an Account, Opportunity, Case, etc.	✗ No	✓ Yes
Both Contact and Opportunity involved in Task	✓ Yes	✓ Yes

Important Notes

- **Only Leads and Contacts** can be assigned to **WhoId**. Other standard/custom objects are **not allowed**.
- **WhatId** can point to any object **except** Lead or Contact.
- Both fields are **optional**, but at least one is usually present for meaningful activity tracking.

Sample Code

apex

```
Task t = new Task(
    Subject = 'Call to follow up',
    Status = 'Not Started',
    Priority = 'Normal',
    WhoId = '0031x00000XXXXXX', // Contact ID
    WhatId = '0061x00000YYYYYY' // Opportunity ID
);
insert t;
```

Let me know if you'd like a **diagram** or **Flow Builder example** that uses `WhoId` and `WhatId`.



what is builder page and what can be done with this

In Salesforce, the **Builder Page** typically refers to tools within **Experience Cloud (formerly Community Cloud)** where you **design, customize, and manage** your Experience Cloud sites (also called portals or digital experiences).

What is the Builder Page?

The **Experience Builder** (or simply **Builder**) is a **drag-and-drop visual design tool** in Salesforce that lets you:

- **Create and customize** branded Experience Cloud sites without writing code.
- Control the **layout, pages, themes, and components** of your site.
- Define **navigation menus**, visibility rules, and **page access** based on user profiles or audiences.

What Can You Do With the Builder Page?

Here's a detailed breakdown:

Feature	Description
Design Pages Visually	Use drag-and-drop UI to create pages, headers, footers, and layout components.
Add Components	Add standard (e.g., Feed, Case List) or custom components (Lightning Components).
Set Page Layouts	Choose from pre-built templates or build custom page structures.
Customize Branding	Change colors, fonts, logos, and themes to match your company branding.
Control Navigation	Add navigation menus, links to other pages, and external websites.
Set Audience Targeting	Show or hide content based on user profile, permission, or audience.
Create Dynamic Pages	Use record-based page variations based on object or URL parameters.
Connect to Salesforce Data	Display records using components tied to standard/custom Salesforce objects.
Manage Themes and Style	Choose from built-in themes or customize using the Theme panel.
Preview and Publish Site	Preview the site for different users and publish updates live.

Example Use Cases

Use Case	How Builder Helps
Customer Portal to log/view cases	Drag in Case List component, customize branding, restrict to customer profiles.

Use Case	How Builder Helps
Partner Portal to register leads	Add custom forms, connect to Lead object, apply audience filters.
Knowledge Base site for external users	Embed KnowlEdge component, create SEO-friendly public pages.
Employee Intranet	Secure internal community with Chatter, announcements, dashboards.

Builder Interface Key Sections

Section	Purpose
Components Panel	Add standard/custom components like Record List, Rich Text, etc.
Page Structure	Modify layout, drag-and-drop components.
Theme Panel	Change color schemes, headers, logos, and branding.
Settings Panel	Configure general settings like languages, URLs, and error pages.
Preview Button	See how your site looks to various user roles before publishing.

Accessing Builder Page

1. Go to **Setup** → **Digital Experiences** → **All Sites**
2. Click **Builder** next to your site
 - You'll be taken to the **Experience Builder**

Related Concepts

Term	Description
Experience Cloud Site	The actual community or portal website you are building
Audience	A set of rules (profile, location, language) to show/hide content dynamically
Theme	Predefined or custom design skin applied to your site
Page Variations	Different versions of a page shown based on record or user data

Summary

The **Builder Page** (Experience Builder) is a **powerful no-code tool** in Salesforce used to:

- Design and manage digital experiences
- Customize site pages with data-aware components
- Control content visibility based on users or conditions
- Maintain your brand and site structure easily

Would you like a **step-by-step example** of building a customer portal with Case functionality?



can you give me step by step guide how to build experience cloud and how to link it with lwc

Absolutely! Here's a **step-by-step guide** to build an **Experience Cloud Site** in Salesforce and **link it with LWC (Lightning Web Components)**.

✓ PART 1: Enable and Set Up Experience Cloud

Step 1: Enable Digital Experiences (Experience Cloud)

1. Go to **Setup**
2. Search for "**Digital Experiences**" or "**Experience Settings**"
3. Click **Enable Digital Experiences**
4. Set your domain name (e.g., `yourcompany.force.com`)
5. Click **Save**

Step 2: Create a New Site

1. Go to **Setup** → **Digital Experiences** → **All Sites**
2. Click **New**
3. Choose a template:
 - **Customer Account Portal**
 - **Partner Central**
 - **Help Center**
 - Or **Build Your Own (LWR)** for LWC-based sites
4. Enter site name and URL
5. Click **Create**

✓ PART 2: Customize Your Site in Experience Builder

Step 3: Open Experience Builder

1. After creation, click **Builder** next to your site
2. You'll enter the **Experience Builder UI**
3. Use the **drag-and-drop interface** to:
 - Add **Components**
 - Add **Pages**
 - Style with **Themes**

- Set Navigation Menu

✓ PART 3: Expose and Use LWC in Experience Cloud

Step 4: Create a Lightning Web Component

In VS Code or Dev Console:

```
bash

sfdx force:lightning:component:create --type lwc --componentname myCommunityComponent --
outputdir force-app/main/default/lwc
```

myCommunityComponent.js

```
js

import { LightningElement } from 'lwc';

export default class MyCommunityComponent extends LightningElement {
  greeting = 'Hello Experience Cloud!';
}
```

myCommunityComponent.html

```
html

<template>
  <h1>{greeting}</h1>
</template>
```

Step 5: Expose the LWC to Experience Cloud

Modify the **meta.xml** file:

myCommunityComponent.js-meta.xml

```
xml

<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>60.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightningCommunity__Page</target>
    <target>lightningCommunity__Default</target>
  </targets>
</LightningComponentBundle>
```

✓ This makes your LWC available on **Experience Builder** pages.

Step 6: Deploy the LWC to Salesforce

If using Salesforce CLI:

```
bash

sfdx force:source:deploy -p force-app/main/default/lwc/myCommunityComponent
```

Step 7: Add LWC to Experience Cloud Page

1. Open **Experience Builder** (Setup → All Sites → [Your Site] → Builder)
2. In the **Components panel**, scroll to **Custom Components**
3. Drag and drop **myCommunityComponent** onto the page
4. Click **Publish** to make it live

✓ PART 4: Assign Permissions for LWC and Site Access

Step 8: Grant Guest/User Access to LWC Apex Classes (if needed)

If your LWC calls Apex:

1. Go to **Setup** → **Apex Classes**
2. Click **Security**
3. Add **Guest User Profile** or any Site Profile needing access

Step 9: Assign a Profile to Users

1. Go to **Setup** → **Users** → **Profiles**
2. Make sure the **Experience Cloud User checkbox** is enabled
3. Assign permissions to access necessary objects and tabs
4. Link the profile to the site from:
 - **Setup** → **All Sites** → **[Site]** → **Workspaces** → **Administration** → **Members**

Optional: Make the LWC Dynamic

You can make your LWC react to record data by using:

```
js

import { LightningElement, api } from 'lwc';

export default class MyCommunityComponent extends LightningElement {
  @api recordId; // auto-passed when using on record pages
}
```

Summary

Step	Action
1. Enable Experience Cloud	Setup → Experience Settings
2. Create Site	Choose template, name, and publish
3. Use Experience Builder	Design pages and layout visually
4. Create and expose LWC	Use lightningCommunity__Page or Default target
5. Add LWC to Site	Drag and drop from Custom Components
6. Assign Permissions	Apex Class/Object access to profiles/guest users

Would you like:

- A **real-world example** of an LWC that displays case status for logged-in users?
- A **diagram** or **project folder structure** to help you organize your code?

Let me know!



PART 2: Customize Your Site in Experience Builder

Step 3: Open Experience Builder

After creation, click Builder next to your site

You'll enter the Experience Builder UI

Use the drag-and-drop interface to:

Add Components

Add Pages

Style with Themes

Set Navigation Menu

You're absolutely right, and let's expand **Step 3** (Customize Your Site in Experience Builder) into a full **hands-on walkthrough**, so you know exactly how to design your site using **Experience Builder**.

PART 2: Customize Your Site in Experience Builder

Step 3: Open Experience Builder

Once your Experience Cloud site is created, follow these steps:

Access the Builder

1. Go to **Setup** → **Digital Experiences** → **All Sites**
2. Find the site you created
3. Click the **"Builder"** link next to your site name

You are now inside the Experience Builder UI

Here's how to **use the interface** to build and customize your site:

What You Can Do in Experience Builder

Feature	Details
---------	---------

✓ 1. Add Components

- Left panel → Drag-and-drop standard or custom components
- Examples:
 - **Record List** – Show a list of Salesforce records
 - **Rich Content Editor** – Add images, text, videos
 - **LWC/Custom Components** – Add your own LWCs you've exposed

 You can also reuse components across pages.

✓ 2. Add Pages

- Click the **"Pages"** dropdown at the top
- Options:
 - **Standard Pages** (Home, Contact Support, Record Detail)
 - **Object Pages** (Case, Contact, Lead, etc.)
 - **Custom Pages** (blank layout to design from scratch)

 You can set page visibility based on **audiences**, such as user roles or profiles.

✓ 3. Style with Themes

- Click the **Theme** icon (paintbrush icon on top right)
- Choose a **prebuilt theme**, or customize:
 - Header
 - Footer
 - Colors
 - Fonts

- Backgrounds
- Logo

You can also create your own **Custom Theme Layouts** using Aura Components.

✓ 4. Set Navigation Menu

- From the top bar, click **Navigation Menu**
- Define links to:
 - Internal pages (e.g., Home, My Cases)
 - External URLs
 - Custom links
- Drag items to reorder
- Use **nested menus** for grouping

 You can also **show/hide menu items** based on user **audience**.

✓ 5. Set Page Access

- Click on the gear icon → **Settings** → **Page Access**
- Define who can access what page
 - Logged-in users
 - Public (for Knowledge Articles or Help Center)
 - Specific profiles

✓ 6. Preview and Publish

- Click **Preview** to test the site for:
 - Logged-in user
 - Guest user
 - Specific profiles
- When ready, click **Publish** to go live.

Pro Tips

Task	How-to
Add a custom LWC	Drag from Custom Components on the left panel
Assign different layouts	Use Page Variations based on user type or record field values
Enable multi-language support	Go to Settings → Languages , add translations
Enable login functionality	Go to Settings → Login & Registration , customize login page

Task	How-to
Redirect after login	Use Login Target Page in site settings

Summary

The **Experience Builder** allows **non-developers** and developers alike to:

You Can...	Using...
Design Pages	Drag-and-drop visual editor
Add LWCs	Expose them with lightningCommunity target
Customize Styling	Theme panel (colors, fonts, headers)
Create Menus & Navigation	Navigation Menu Manager
Set Page Rules	Audience and Visibility Rules
Publish Changes	One-click Publish button

Would you like a **guided checklist** to build a customer portal for submitting and viewing cases?

Or would you like to **automate user creation** for Experience Cloud using Flow or Apex next?